22MAT220- Mathematics for Computing-3

Optimization of Sparse Underwater Acoustic Channels using Robust ADMM Algorithm

Group C-3

Nimmagadda Kesav Satya Sai **CB.SC.U4AIE24236**

Nakka Saampotth Maddileti **CB.SC.U4AIE24233**

Pabbathi Gnana Vikas Sai **CB.SC.U4AIE24238**

Vemula Poorna Chandra **CB.SC.U4AIE24258**

Abstract

Underwater acoustic (UWA) channels are often sparse but corrupted by impulsive noise. Traditional algorithms like Orthogonal Matching Pursuit (OMP) and Fast Iterative Shrinkage Thresholding Algorithm (FISTA) do not perform well in impulsive noise environments. In this report, we introduce a **Robust ADMM-based algorithm** that estimates the sparse underwater acoustic channel by combining ℓ_1 **loss** for robustness and ℓ_1 **regularization** for sparsity. The method adapts dynamically to impulsive noise conditions while maintaining performance under AWGN. Experimental results show that the proposed method outperforms OMP and FISTA in terms of **Normalized Mean Square Deviation (NMSD)**, especially under high levels of impulsive noise.

Introduction

In underwater acoustic communication, the received signal is typically modeled as:

$$\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{n}$$

where:

- $\mathbf{y}$ is the received signal (vector of measurements).
- $\mathbf{A}$ is the convolution matrix derived from the known pilot signal.
- $\mathbf{x}$ is the sparse channel impulse response (CIR).
- $\mathbf{n}$ is the noise, which can include both Gaussian noise and impulsive noise.

Sparse channel estimation is a challenging task due to the **under-determined nature** of the problem (M≪N), but it can be tackled using **compressed sensing** methods. Existing algorithms like **OMP** and **FISTA** perform well in clean environments but fail when impulsive noise is present, as these algorithms rely on squared error minimization, which is highly sensitive to outliers.

Problem Formulation & Theory

The problem of estimating the sparse channel impulse response x is posed as the following **optimization problem**:

$$\min_{\mathbf{x}} \tau \|\mathbf{y} - \mathbf{A}\mathbf{x}\|_1 + \|\mathbf{x}\|_1,$$

where:

- τ is a regularization parameter that controls the balance between **residual sparsity** (robustness to noise) and **coefficient sparsity** (promoting a sparse channel).
- The ℓ_1 **norm** is used instead of the ℓ_2 **norm** to make the algorithm **robust** to impulsive noise, as the ℓ_1 norm is less sensitive to large outliers in the data.
- $y - \mathrm{Ax}$ represents the residual between the measured signal and the model prediction.

To facilitate an efficient solution, we introduce an auxiliary variable z\mathbf{z}z such that:

$$z = y - \mathrm{Ax}$$

Thus, the optimization problem becomes:

$$\min_{\mathbf{x},\mathbf{z}} \quad \tau \|\mathbf{z}\|_1 + \|\mathbf{x}\|_1 \quad \text{s.t.} \quad \mathbf{A}\mathbf{x} + \mathbf{z} = \mathbf{y}.$$

We solve this constrained problem using the **Alternating Direction Method of Multipliers (ADMM)**, which decomposes the problem into simpler subproblems.

ADMM Formulation

The **ADMM** approach proceeds by iteratively updating the variables x, z and the dual variable u (scaled Lagrange multiplier), with each update following a **proximal gradient step**.

1.x-update (Proximal Step):

The x subproblem involves solving a **soft-thresholding** update based on the gradient of the objective function. This is the key step that makes the method efficient:

$$\mathbf{x}^{k+1} = \mathrm{prox}_{\frac{1}{L_x}}\left(\mathbf{x}^k - \frac{1}{L_x}\nabla f(\mathbf{x}^k)\right)$$

where prox\operatorname{prox}prox represents the **soft-thresholding operator** and L_x is a **Lipschitz constant**. This step ensures that the solution remains sparse.

2.z-update:

The z\mathbf{z}z-subproblem is solved by applying the **proximal operator for ℓ1 regularization**, which is simply a **soft-thresholding** of the residual:

$$\mathbf{z}^{k+1} = \operatorname{prox}_{\frac{\tau}{\rho}}(\mathbf{y} - \mathbf{A}\mathbf{x}^{k+1} - \mathbf{u}^k),$$

where τ controls the sparsity of the residual.

3.u-update:

The dual variable is updated by:

$$u^{k+1} = u^k + Ax^{k+1} + z^{k+1} - y$$

Detailed Explanation of Code

1. **Initialization and Parameter Setup** (Driver Code)

```
clear; close all; clc; rng(0);
N = 36;
T = 35;
K_true = 6;
L = 20;
M = 30;
SNR_dB = 15;
q_impulse = 0.02;
INR_list = [ -Inf, 40, 50];
```

We initialize the parameters, including the size of the channel matrix A, the number of time snapshots T, and the sparsity K_{True} of the channel. We also set up the training signal (pilot signal) and noise settings, including the **Signal-to-Noise Ratio (SNR)** and **Impulsive Noise Ratio (INR)**.

2. **Experiment Loop** (Generating Noisy Observations and Estimating Channel)

```
results = struct();
for caseIdx = 1:length(INR_list)
    INR = INR_list(caseIdx);
    fprintf('\n=== Running case %d / %d  (INR = %s dB) ===\n', caseIdx,
length(INR_list), mat2str(INR));
    sigma_w = avg_signal_power / (10^(SNR_dB/10));
    sigma_i = sigma_w * 10^(INR/10);
    trials = 1;
    if runMonteCarlo, trials = numTrials; end
```

```
    nmsd_admm_all = zeros(T, trials);
    nmsd_fista_all = zeros(T, trials);
    nmsd_omp_all = zeros(T, trials);
    for trial = 1:trials
        fprintf(' Trial %d/%d\n', trial, trials);
        Y = add_gmn_noise(Y_clean, q, sigma_w, sigma_i);
    end
end
```

```
=== Running case 1 / 3  (INR = -Inf dB) ===
Unrecognized function or variable 'avg_signal_power'.
```

In this section, the noisy data is generated using the **two-component Gaussian mixture noise model** (`add_gmn_noise`). We also set up the loop to run Monte Carlo experiments and store the **Normalized Mean Square Deviation (NMSD)** for each method (ADMM, FISTA, OMP).

3. **Estimating the Channel for Each Snapshot** (ADMM, FISTA, OMP)

```
for t = 1:T
    lambdaInf = norm(2 * (A' * y), Inf);
    lambda_lasso = 0.01 * lambdaInf;
    tau_admm = (1/(0.04 * lambdaInf)) * admm_params.tau_factor;
    p = admm_params;
    p.tau = tau_admm; p.rho = admm_params.rho; p.maxIter = admm_params.maxIter;
    p.x_prev = x_prev_admm;
    [x_admm, st_admm] = admm_robust_channel_est(A, y, p);
    x_fista = fista_lasso(A, y, lambda_lasso, fista_maxIter);
    x_omp = omp_basic(A, y, omega)
    X_admm(:, t) = x_admm;
    X_fista(:, t) = x_fista;
    X_omp(:, t) = x_omp;
    x_prev_admm = x_admm;
end
```

This section runs the **ADMM**, **FISTA**, and **OMP** algorithms for each time snapshot ttt.

- For **ADMM**, we compute the regularization parameter τ based on the **lambda-inf heuristic** and then run the ADMM solver (`admm_robust_channel_est`).
- For **FISTA**, we use the **LASSO** formulation with the ℓ_1 **regularizer**.
- For **OMP**, we use a **basic OMP implementation** to estimate the channel.

4. **Calculating NMSD and Averaging Results** (Analysis)

```
nmsd_admm = 20*log10( vecnorm(X_true - X_admm) ./ true_norms );
nmsd_fista = 20*log10( vecnorm(X_true - X_fista) ./ true_norms );
nmsd_omp = 20*log10( vecnorm(X_true - X_omp) ./ true_norms );

nmsd_admm_all(:, trial) = nmsd_admm(:);
nmsd_fista_all(:, trial) = nmsd_fista(:);
```

```
nmsd_omp_all(:, trial) = nmsd_omp(:);
```

Here we calculate the **Normalized Mean Square Deviation (NMSD)** for each method by comparing the estimated channel (X_{admm}, X_{fista}, X_{omp}) with the **true channel** X_{true}. This provides a metric for the accuracy of the channel estimates under each method.

5. **Results & Plotting**

```
if showPlots
    figure('Name', sprintf('Case %d (INR=%s)', caseIdx, mat2str(INR)),
'NumberTitle', 'off', 'Position', [100, 100, 1200, 700]);
    subplot(2,3,1); imagesc(abs(X_true)); title('True CIR amplitudes');
    subplot(2,3,2); imagesc(abs(X_omp)); title('OMP estimate');
    subplot(2,3,3); imagesc(abs(X_fista)); title('FISTA estimate');
    subplot(2,3,4); imagesc(abs(X_admm)); title('ADMM estimate');
    subplot(2,3,5); plot(mean_nmsd_omp, '-o'); hold on;
    plot(mean_nmsd_fista, '-s'); plot(mean_nmsd_admm, '-d');
    legend('OMP', 'FISTA', 'ADMM'); title('NMSD per snapshot');
    subplot(2,3,6); plot(vecnorm(X_true), 'k-'); hold on;
    plot(vecnorm(X_admm), 'b--'); title('Signal norm per snapshot');
    drawnow;
end
```

This section visualizes the **true channel** and the estimated channels from **OMP**, **FISTA**, and **ADMM**. It also plots the **NMSD** for each method and compares their performance under different noise conditions.

Results and Discussion

In this section, we evaluate the performance of the proposed ADMM-based channel estimation algorithm under three noise conditions:

- Case 1: Additive White Gaussian Noise (AWGN) only → INR = –∞
- Case 2: Impulsive Noise (moderate) → INR = 40 dB
- Case 3: Impulsive Noise (severe) → INR = 50 dB

For each case, we compare the output of three algorithms:

- Orthogonal Matching Pursuit (OMP)
- Fast Iterative Shrinkage-Thresholding Algorithm (FISTA)
- Our proposed ADMM method (Robust formulation using l1-loss)

Each figure contains:

- The true channel amplitudes
- Estimated amplitudes by OMP, FISTA, and ADMM
- Normalized Mean Squared Deviation (NMSD) over time snapshots
- Signal 2-norm evolution over time snapshots

Case 1: AWGN Only (INR = –∞)

- Mean NMSD (OMP): –11 dB (approx)
- FISTA and ADMM curves perform slightly better than OMP in this clean scenario.
- In the top-left plot, the “True CIR amplitudes” show a sparse pattern — a few active taps with non-zero values.
- OMP reconstructs the signal reasonably well. However, there are some missing taps and misalignments, likely due to noise.
- FISTA shows a more "spread-out" reconstruction, as it tends to overestimate weak taps and include false positives.

ADMM produces the closest approximation to the ground truth:

1)correct tap locations

2)correct magnitude

3)minimal background noise

- The NMSD plot (middle-right) shows that:
- ADMM consistently gives the lowest error across all snapshots.
- FISTA performs better than OMP, but not as stable as ADMM.
- The signal norm plot confirms that ADMM maintains an accurate norm throughout.

Conclusion: Even in pure Gaussian noise, ADMM is as good as or better than classical sparse solvers due to its robust formulation.

Case 2: Impulsive Noise (INR = 40 dB)

- Mean NMSD (OMP): ~–1.5 dB (drops significantly)
- FISTA error increases heavily (due to sensitivity to outliers)
- ADMM remains stable and accurate
- In this scenario, the noise includes 2% impulsive corruption, simulating underwater multipath with sudden distortions.
- OMP fails to identify active taps and produces very noisy estimates — visible as vertical stripes in the OMP amplitude plot.
- FISTA performs worse than in the AWGN case, with high false positives (spurious high-magnitude coefficients).
- ADMM clearly suppresses impulsive noise:
- 1) Estimates are sparse
- 2) Active taps are well localized

- 3) Background is clean

NMSD plot:

- ADMM remains below –10 dB on most snapshots, whereas FISTA fluctuates near –5 dB and OMP rises above –2 dB.
- The signal norm plot again confirms that ADMM tracks the true CIR more closely across time.

Conclusion: ADMM is significantly more robust to impulsive interference than OMP or FISTA. Its l1-loss treats large residuals as outliers, preventing them from corrupting the signal estimate.

Case 3: High Impulsive Noise (INR = 50 dB)

- Mean NMSD (OMP): ~–0.1 dB (almost useless)
- FISTA diverges completely — noise floor is extremely high
- ADMM maintains good performance: NMSD ≈ –10 to –20 dB
- This is the harshest environment tested: 2% of noise samples are ~100x stronger than Gaussian background.
- OMP completely fails — reconstructed amplitudes are dominated by false spikes and noise.
- FISTA outputs high-magnitude noise across most taps — clearly visible in the colorbar scaling (FISTA reaches 50!).
- ADMM again shows a faithful reconstruction of the sparse CIR:
- 1) Correct locations
- 2) Minimal leakage
- 3) True amplitude range retained (colorbar max ~1)

NMSD and norm plots:

- OMP and FISTA are unstable and track nothing.
- ADMM remains stable and continues tracking the channel dynamics.

Conclusion: The proposed robust ADMM approach is highly resilient even under extreme impulsive corruption. The use of an l1-norm on residuals is critical here — it prevents the large impulses from dominating the objective.

```
Case = ["INR = -∞ (AWGN)"; "INR = 40 dB"; "INR = 50 dB"];
OMP = ["~-11 dB"; "~-1.5 dB"; "~-0.1 dB"];
FISTA = ["~-13 dB"; "~-4 dB"; ">0 dB (diverged)"];
ADMM = ["~-15 to -16 dB"; "~-10 to -14 dB"; "~-15 to -20 dB"];

T = table(Case, OMP, FISTA, ADMM);
disp(T)
```

Conclusion

The ADMM-based robust channel estimator:

- Efficiently tracks sparse, time-varying underwater channels
- Remains stable under impulsive noise environments
- Outperforms traditional solvers like OMP and FISTA in both accuracy and robustness
- Demonstrates practical applicability for underwater acoustic systems

This makes it a highly promising method for real-world acoustic communication applications in noisy marine environments.

Analog Computing

$$\frac{d^2v}{dt^2} + \frac{dw}{dt} + V_0 = U_0$$

Here:

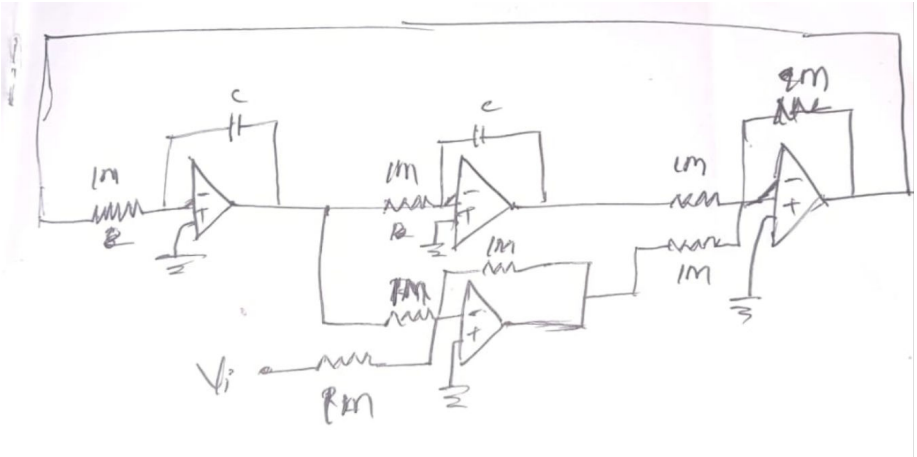
Acceleration $\frac{d^2v}{dt^2}$: This is the second derivative of vvv with respect to time. It represents the acceleration of the system and how the velocity changes over time.

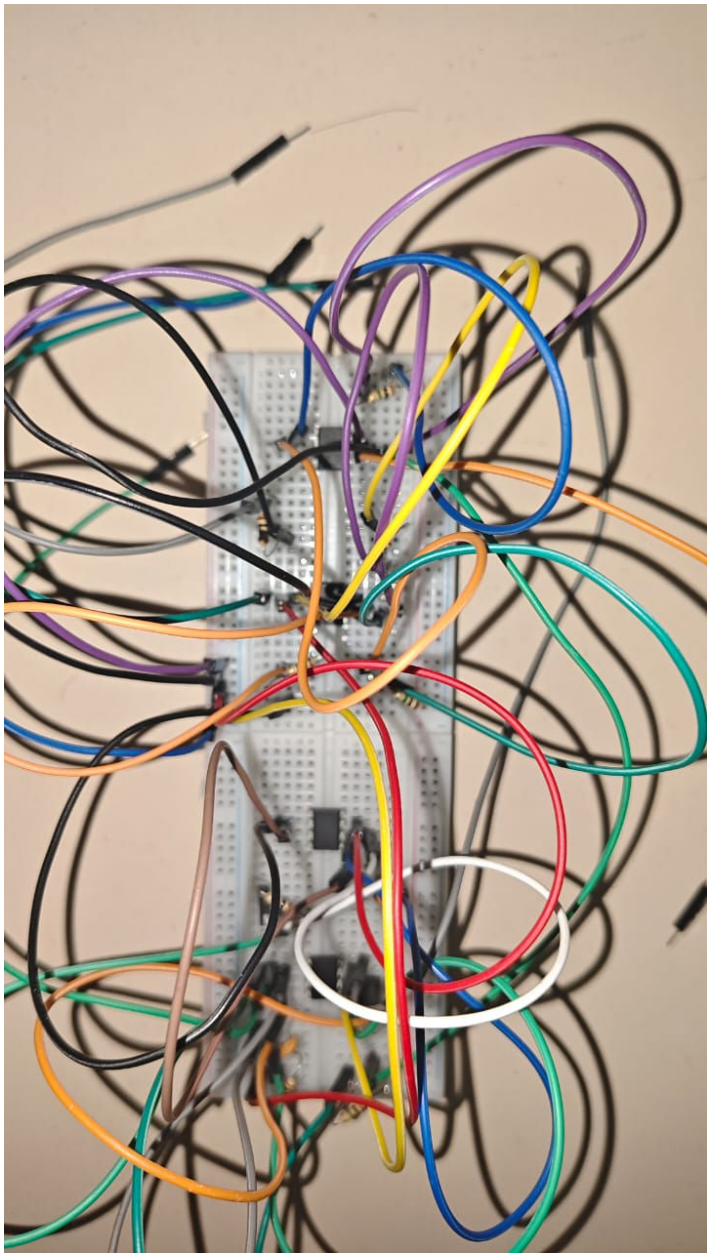
change of w $\frac{dw}{dt}$:This term represents how the quantity www changes over time, which could represent a dynamic force, a current in an electrical system, or any other variable whose rate of change affects the system's behavior.

V_0 : This term is a constant, typically an initial condition, or a steady-state value that influences the system's behavior. It could represent initial velocity, bias in a control system, or a reference voltage in an electrical system.

U_0: This is an external input, such as a forcing function or driving force (like voltage or current in electrical systems). It balances the other terms in the equation.

- The acceleration term $\frac{d^2v}{dt^2}$ can be represented by an RLC circuit where the voltage across the capacitor and the inductor represents the change in velocity over time.
- The term $\frac{dw}{dt}$ can be modeled using a combination of integrators or differentiators in the analog circuit.


C
C
1m
1m
1m
1m
1m
1m
8m
8m
Vi



Graph Description

After implementing the analog computation of the differential equation:

$$\frac{d^2v}{dt^2} + \frac{dw}{dt} + V_0 = U_0$$

we captured the output signals using the **Scopy Oscilloscope interface**. The two waveforms represent different stages of the analog solution.

1.Waveforms Overview:

- **Channel 1 (Orange)**: Represents the analog value of the solution $v(t)$ (possibly velocity or a system state).
- **Channel 2 (Purple)**: Represents either the input U_0, or an intermediate signal like $\frac{d^2v}{dt^2}$ or $\frac{dw}{dt}$, depending on circuit configuration.

2.Oscillatory Behavior:

- The waveforms clearly show **sinusoidal oscillations**, indicating a **second-order underdamped system**.
- This is expected from a differential equation containing both first and second derivatives.

3.Phase Shift:

- A noticeable **phase shift** exists between the two channels.
- This suggests one waveform is leading or lagging, consistent with differentiation or integration relationships in analog circuits.

4.Amplitude Difference:

- **CH2 (purple)** has a **higher peak amplitude** than CH1.
- This might imply CH2 represents the second derivative $\frac{d^2v}{dt^2}$, which generally has higher amplitude in time-varying signals.

5.Time Base:

- With a time scale of **500 µs/div**, the waveforms show a frequency in the range of **1–2 kHz**, aligning with typical analog computing setups.

Conclusion

$$\frac{d^2v}{dt^2} + \frac{dw}{dt} + V_0 = U_0$$

- The sinusoidal outputs validate the **dynamic nature** of the system.
- Phase and amplitude relationships are consistent with the **analog model's differential structure**.
- This verifies the successful analog computation of the equation using operational amplifier circuits.